

COS30017 - Software Development for Mobile Devices

Assignment 3: Novel Library Application

Nguyễn Chí Cường - 104852250



Git repository: <https://github.com/Opatzo2212/COS30017.git>

1. Introduction & Application Overview

My application is a Novel Library that is built to track and save any novels or books that the user wants to read. The application allows users to persistently store, view, update, and manage a personal library of novels. Key functionalities include creating and editing novels, a grid-based library view with real-time search and sorting (A-Z, Recent), local image persistence, and an interactive reading status tracker (Plan to Read, Reading, Completed, Dropped).

2. About the project

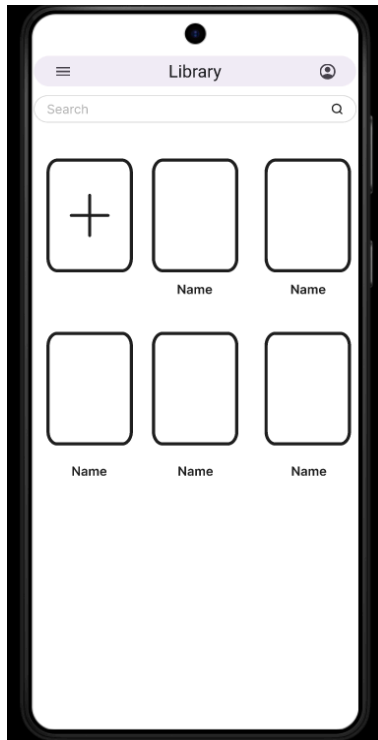
2.1. Project timeline

The application was developed over the course of 2 weeks.

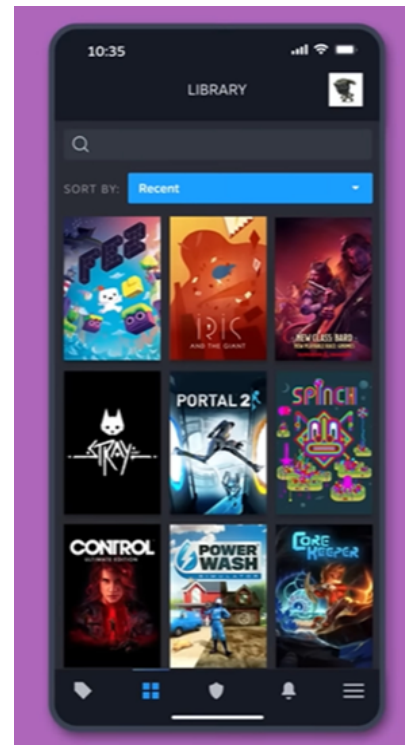
- **Week 1:** During the first week I have delved into research and drafting the core structure of the app. The first thing I did was to look into a suitable database to implement.

While raw SQLite was considered, Room was chosen to be my database for its compile-time

verification and seamless integration with LiveData. I looked at design choices for the app and built a figma model for the app by taking inspiration from other libraries.



The figma UI

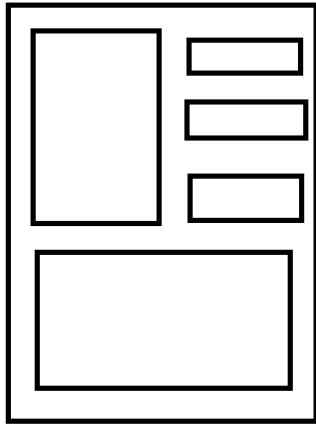


App inspiration

For the architectural planning, the Model-View-ViewModel architecture was mapped out to separate the user interface from the data logic. This involved designing the Book data entity to accommodate future features like reading status and chapter tracking.

Basic interfaces I considered were a log in and sign up activity, the main library activity and smaller fragments inside it. Considerations for mobile hardware restrictions, such as utilizing ConstraintLayout and XML Barrier elements, were planned during this phase to prevent UI overlap on smaller devices

- **Week 2:** The second week is when I started coding the app. I first set up the main UI for the app to be used. This includes the basic interface mentioned above. For smaller fragments I create a draft version to follow.



Ui for fragment

Setting up the data layer was the next step. The Room Database, DAOs, and Repositories were implemented first. Background coroutines were established in the BookViewModel to handle database I/O operations without blocking the main UI thread. Afterward I spent time debugging and testing features to ensure they work smoothly and without any bugs.

2.2. Project implementation

For this project, the implementation required moving beyond basic Android components to establish a scalable, production-ready architecture. Rather than simply connecting UI elements to static data as might have been acceptable in previous assignments, this application was built to align with modern Android development standards, specifically emphasizing separation of concerns and lifecycle awareness.

In previous tasks, it was common to handle data manipulation directly inside the UI controllers. However, for this project, I explicitly adopted the Model-View-ViewModel architecture. Coupling database queries directly to the UI leads to memory leaks and app crashes during configuration changes. By introducing a ViewModel, the UI data survives these lifecycle events. Furthermore, utilizing a Repository pattern creates a single source of truth, abstracting the data origin away from the UI. This means if we later migrated from a local database to a cloud API, the UI layer would not need to be rewritten.

```

class BookViewModel(application: Application) : AndroidViewModel(application) {

    private val repository: BookRepository

    init {
        val bookDao = BookDatabase.getDatabase(context = application).bookDao()
        repository = BookRepository(bookDao)
    }

    fun getAllBooksForUser(userId: Int): LiveData<List<Book>> {
        return repository.getAllBooks(userId)
    }

    fun insertBook(book: Book) = viewModelScope.launch(context = Dispatchers.IO) {
        repository.insert(book)
    }

    fun updateBook(book: Book) = viewModelScope.launch(context = Dispatchers.IO) {
        repository.update(book)
    }

    fun deleteBook(book: Book) = viewModelScope.launch(context = Dispatchers.IO) {
        repository.delete(book)
    }

    fun getBookById(id: Int): LiveData<Book> {
        return repository.getBookById(id)
    }
}

```

As for the database, while raw SQLite is functional, it lacks compile-time verification. Room maps our User and Book Kotlin data classes directly to SQLite tables using annotations, catching SQL syntax errors during compilation rather than crashing at runtime. To handle database operations, Kotlin Coroutines were implemented. Modern Android practice strictly prohibits database operations on the Main UI thread to prevent Application Not Responding errors. Coroutines were chosen because it provide a lightweight, native way to execute asynchronous code sequentially.

```

package com.example.assignments3

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "books")
data class Book(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val userId: Int,
    val title: String,
    val author: String,
    val tags: String,
    val link: String,
    val description: String,
    val coverImageUri: String?,
    val status: String = "Plan to Read",
    val currentChapter: Int = 0
)

```

```

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Delete
import androidx.room.Update

@Dao
interface BookDao {
    @Insert(onConflict = OnConflictStrategy.IGNORE)
    suspend fun insert(book: Book)

    @Query(value = "SELECT * FROM books WHERE userId = :userId ORDER BY id DESC")
    fun getAllBooks(userId: Int): LiveData<List<Book>>

    @Query(value = "SELECT * FROM books WHERE id = :id")
    fun getBookById(id: Int): LiveData<Book>

    @Delete
    suspend fun delete(book: Book)

    @Update
    suspend fun update(book: Book)
}

```

I have created the application to utilize a hybrid navigation model with distinct Activities for login(LoginActivity, SignUpActivity), but a Single-Activity architecture (MainActivity) relying on Fragments for the core library experience.

```

<FrameLayout
    android:id="@+id/fragmentContainer"
    android:layout_width="0dp"
    android:layout_height="0dp"
    app:layout_constraintTop_toBottomOf="@id/sortLayout"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

3. Testing, Debugging, and Problem Solving

By using Logcat to debug I managed to find bugs that I did not notice.

```

Log.d( tag = "AuthDebug", msg = "Attempting login for email: $email")

userViewModel.loginUser(email, password) { user ->
    if (user != null) {
        Log.i( tag = "AuthDebug", msg = "Login SUCCESS: User ID ${user.id}, Name: ${user.name}")

        val sharedPreferences = getSharedPreferences( name = "UserSession", mode = Context.MODE_PRIVATE)
        sharedPreferences.edit {
            putInt( p0 = "USER_ID", p1 = user.id)
            .putString( p0 = "USER_NAME", p1 = user.name)
        }

        val intent = Intent( packageContext = this, cls = MainActivity::class.java)
        startActivity(intent)
        finish()
    } else {
        Log.e( tag = "AuthDebug", msg = "Login FAILED: Invalid credentials for $email")
        Toast.makeText( context = this, text = "Invalid email or password", duration = Toast.LENGTH_SHORT).show()
    }
}

```

Data Loss Bug: During the testing phase for adding novels I noticed the data for description did not save properly unlike the rest of the data. When saving a novel, the description field was recorded as blank in the Room database, despite the user explicitly typing data into the interface.

- Debugging process: By adding a log into the btnSave listener to print the extracted descriptionText at the exact moment of execution. The log returned an empty string, proving the database was functioning correctly, but the UI extraction was failing.
- Solution: A thorough inspection of the XML layout revealed a duplicate @+id/etDescription tag hidden beneath the main layout. Removing the duplicate ID completely resolved the data extraction failure.

4. Use case and user stories

4.1. Use cases

- Use case 1: Adding a novel. Users can click on the add novel button to open the add novel fragment. There, users can input a novel's metadata such as Title, Author, Description, Tags, Link. Users can also upload image from their device gallery for the novel cover. Clicking save novel will save the novel's data and the novel will appear in the main library screen.

- Use case 2: Keeping track of reading status. Users can click on the novel to view their detail and reading status. Users can edit the reading status by choosing between Plan to Read, Reading, Completed, Dropped.

4.2. User Stories

- User Story 1: "As someone who's sharing a device with my family, I wanted a place to keep note of my novels library without my family looking at it. This Novel Library app allows me to note all my stories in a place my family wouldn't see."
- User Story 2: "As an avid reader, I want to manually customize my novel's details so that my library is visually distinct and catered to my preferences."

5. Reflection

5.1. What worked well

The decision to heavily invest time during Week 1 in architectural planning and UI drafting paid significant dividends during the coding phase. Adopting the MVVM architecture, while initially presenting a steeper learning curve compared to previous assignments, ultimately made the application much more stable. Separating the UI controllers from the data logic meant that managing state changes—such as updating a novel's reading status or incrementing chapters—did not lead to UI freezing or memory leaks.

Additionally, the hybrid navigation model (using separate Activities for authentication and Fragments for the core library) proved to be highly effective. It provided a strict security boundary for user sessions while maintaining the lightweight, seamless navigation expected of modern Android applications when browsing the library grid. Using Figma to plan the constraints and barriers beforehand also ensured that the UI scaled predictably across different emulator screen sizes.

5.2. What could be improved

Reflecting on the debugging process, particularly the "Silent Failure" data loss bug caused by a duplicate XML ID, I realized that my early UI file organization could be improved. Establishing a stricter, standardized naming convention for all views (e.g., explicitly prefixing all detail view elements distinctly from add/edit view elements) would prevent these types of overlapping tag issues in the future.

Furthermore, while the current real-time search and sorting functions work perfectly for a moderately sized personal library, loading all database entries into memory at once could become a performance bottleneck if a user tracks thousands of novels. In a future iteration, implementing the Android Paging 3 library would allow the RecyclerView to load data in chunks, further optimizing memory usage. Finally, integrating a network library like Retrofit to automatically fetch novel metadata and cover images via a URL scrape would significantly enhance the user experience by reducing manual data entry.

6. Conclusion

This assignment has been relatively difficult for me. Having to come up with an app on my own and implementing the database was something that took me a long time to learn. Still I would say my app has robust database management through Room, complex XML layout constraints for varying hardware, and effective debugging to ensure smooth user experience.